

Curso online: **Programación orientada a objetos: Java avanzado**

Servlets y JSP

Autores

Aurelio García Prieto

Miguel Sallent Sánchez

ÍNDICE

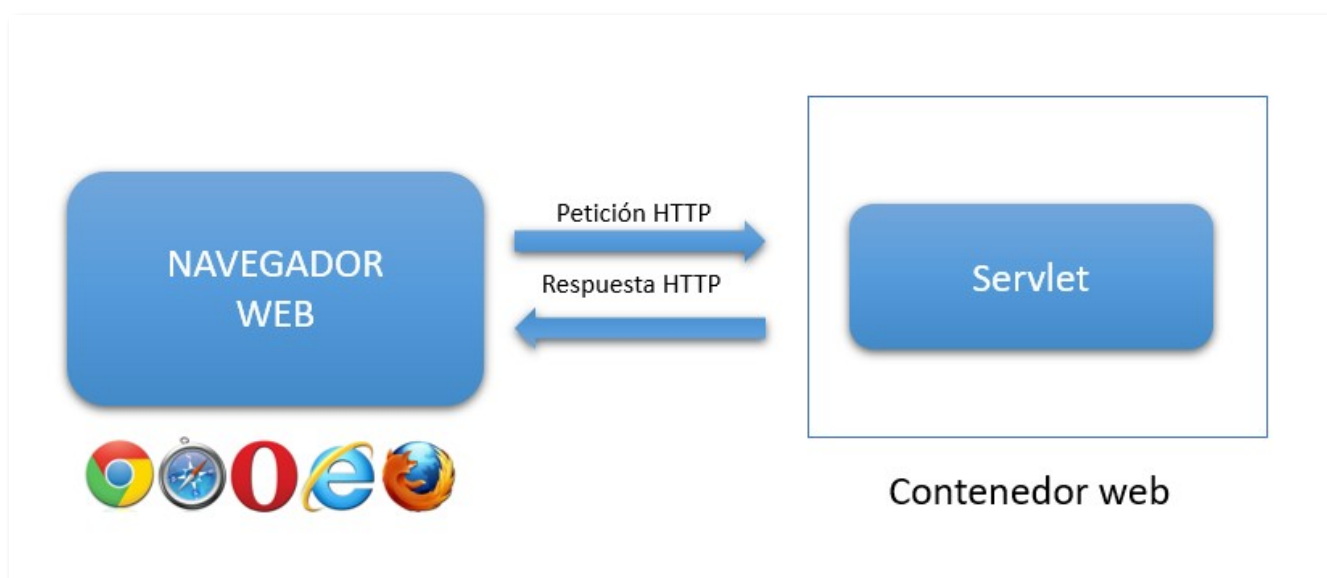
1. Servlets.....	3
2. Ejemplos de Servlets.....	5
3. JavaServer Pages (JSP).....	7
4. Ejemplos de JSP.....	10
5. Cómo crear un proyecto web Dinámico.....	11
6. Ejemplo – MVC.....	19

1. Servlets

Un servlet es una clase en Java que amplía la capacidad de repuesta de un servidor web, permitiendo responder a cualquier tipo de solicitud. Desde peticiones a servicios web a páginas HTML.

Como ya habíamos indicado, un servidor web se limita a responder a peticiones http con contenidos estáticos. Gracias a los servlets se pueden ofrecer distintas respuestas en los servidores web.

Los servlets se cargan o despliegan en un contenedor web que los ejecuta cuando llegan las peticiones para poder contestarlas.



El ciclo de vida de un servlet es el siguiente:

Cuando al contenedor web le llega una petición que está asociada un servlet, el contenedor web:

- Comprueba si hay una instancia del servlet en ejecución. En el caso de no estar ejecutándose:
 - o Carga el servlet.
 - o Crea una instancia de dicha clase.
 - o Inicializa la instancia llamando a su método `init`.
- Si ya hay una instancia del servlet en el contenedor se invoca al método `service` pasando por parámetros los objetos de la petición y de la respuesta.

Por otro lado, al igual que todos los objetos en JAVA, existe un mecanismo para destruirlos y liberar memoria. Si es necesario eliminar el servlet, el contenedor lo finaliza llamando al método `destroy`.

Existen dos tipos de servlets:

- Los genéricos que permiten procesar cualquier tipo de petición. Heredan de la clase *jakarta.servlet.GenericServlet*. Gracias a estos servlets se pueda dar respuesta a peticiones de otros protocolos de internet: SMTP, CGI, FTP, HTTP...
- Y aquellos que implementan el protocolo http. Que heredan de la subclase *jakarta.servlet.http.HttpServlet*. Que únicamente operan con el protocolo http.

Estos últimos son los servlets habituales en las aplicaciones web, ya que operan con el protocolo http, y serán aquellos en los que nos centremos en este curso.

La principal diferencia es que, mientras que los primeros trataban las peticiones de una forma más genérica a través del método service, en el caso de los servlets por http se tratan con métodos específicos para el protocolo http (*get*, *post*, etc.), que es el que emplean los navegadores de internet.

Antes de continuar vamos a realizar una pequeña introducción de cómo funciona el protocolo http.

Protocolo http.

El protocolo http (Protocolo de transferencia de hipertexto) es un protocolo de comunicación que permite por ejemplo las descargas de páginas web, incluyendo las páginas html y otros recursos como imágenes, hojas de estilo, etc.

HTTP es un protocolo sin estado, que no guarda ninguna información sobre las conexiones anteriores. De modo que si una aplicación web necesita identificar al cliente para no confundir los distintos usuarios que interactúan en un momento dado, debe hacer uso de parámetros o *cookies*, que es información que un servidor puede almacenar en el sistema cliente. Gracias a las cookies las aplicaciones web permiten mantener la sesión de los usuarios.

El protocolo http tiene definido una serie de peticiones, pero las más comunes son las siguientes:

- **Get.** Es cuando se solicita un recurso específico desde el cliente. Se utiliza para recuperar información, por ejemplo, para solicitar una imagen o un archivo en concreto del servidor, que podría ser una página html.
- **Post.** Envía datos para que sean procesados en la petición. Generalmente se asocia a los formularios de los navegadores, que envían los campos que lo componen a través de una petición post al servidor.

El protocolo http no sólo describe los métodos para invocar las peticiones. También describe las posibles respuestas que da el servidor a través de códigos de respuesta. Las más conocidas son:

- **Códigos con formato 2xx:** Respuestas correctas. Indica que la petición ha sido procesada correctamente.
- **Códigos con formato 4xx:** Errores causados por el cliente. Indica que ha habido un error en el procesamiento de la petición a causa de que el cliente ha hecho algo mal.
- **Códigos con formato 5xx:** Errores causados por el servidor. Indica que ha habido un error en el procesamiento de la petición a causa de un fallo en el servidor.

Todos habréis visto alguna vez en el navegador un “Error 500”, que se da cuando se ha producido un error en el servidor, o el típico “Error 404”, que se da cuando no se encuentra disponible el recurso solicitado.

Y ¿cómo funcionan los servlets?

Una vez hemos realizado una breve introducción del protocolo http, vamos a explicar cómo funcionan los servlets que operan con dicho protocolo.

Los servlets deberían implementar los métodos `doGet` y/o `doPost` para responder a las peticiones GET y POST que hemos descrito anteriormente.

La estructura básica es:

```
import jakarta.servlet.*;
import jakarta.servlet.http.*;

public class ClaseServlet extends HttpServlet
{
    public void doGet(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException
    {
        // ... código para una petición GET
    }

    public void doPost(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException
    {
        // ... código para una petición POST
    }
}
```

Como se puede observar un servlet es una clase que hereda de `HttpServlet` y sobrescribe los métodos `doGet` y `doPost`. Ambos métodos tienen dos parámetros que representan la entrada y la respuesta del servlet y que se ajustan a las interfaces:

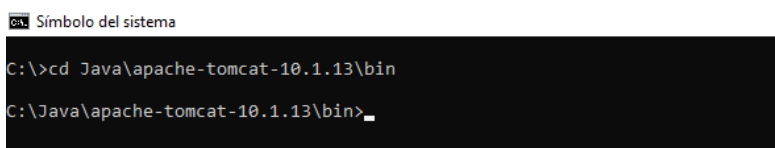
- `HttpServletRequest`. Define los métodos que debe proveer el objeto que representa la **petición** al servlet.
- `HttpServletResponse`. Define los métodos que debe proveer el objeto que representa la **respuesta** del servlet.

2. Ejemplos de Servlets

Una aplicación muy útil que se distribuye con Tomcat es **examples**, donde aparecen ejemplos de servlets y jsp.

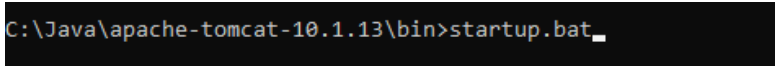
Arrancamos el Tomcat siguiendo las instrucciones del tema anterior:

- Abrir una consola de comandos con el comando `cmd`
- Ubicarse en el directorio `bin`.



```
Símbolo del sistema
C:\>cd Java\apache-tomcat-10.1.13\bin
C:\Java\apache-tomcat-10.1.13\bin>_
```

- Arrancar el Tomcat por medio de la sentencia `startup.bat`.

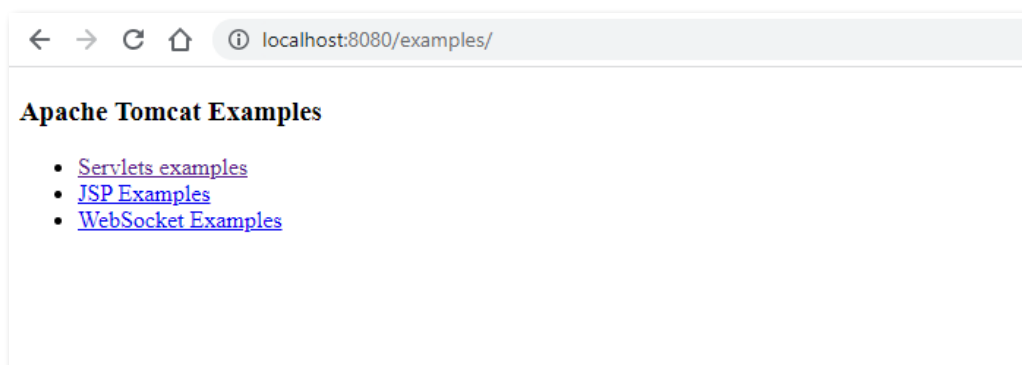


```
C:\Java\apache-tomcat-10.1.13\bin>startup.bat_
```

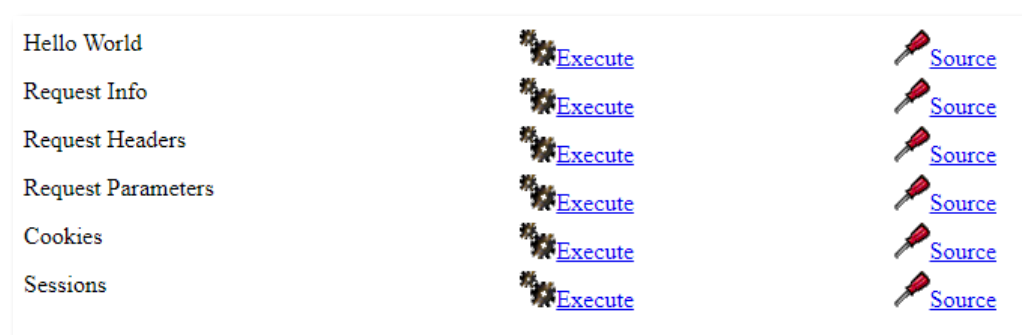
Y accedemos a la url:

<http://localhost:8080/examples>

Podremos ver la siguiente pantalla:



Al acceder a **Servlets examples** aparecen ejemplos sencillos de servlets con su propio código:



Vamos a analizar el ejemplo de Hello World:

Al ejecutarlo se puede observar el típico mensaje de Hola Mundo, y si accedemos al código vemos que es una pequeña clase que encaja con la estructura básica anteriormente descrita. Hereda de `HttpServlet` y sobrescribe el método `doGet`.

Este método escribe en la respuesta (`HttpServletResponse response`) el código en html que deseamos enviar al navegador, el cual lo interpreta como tal.

```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;

public class HelloWorld extends HttpServlet {

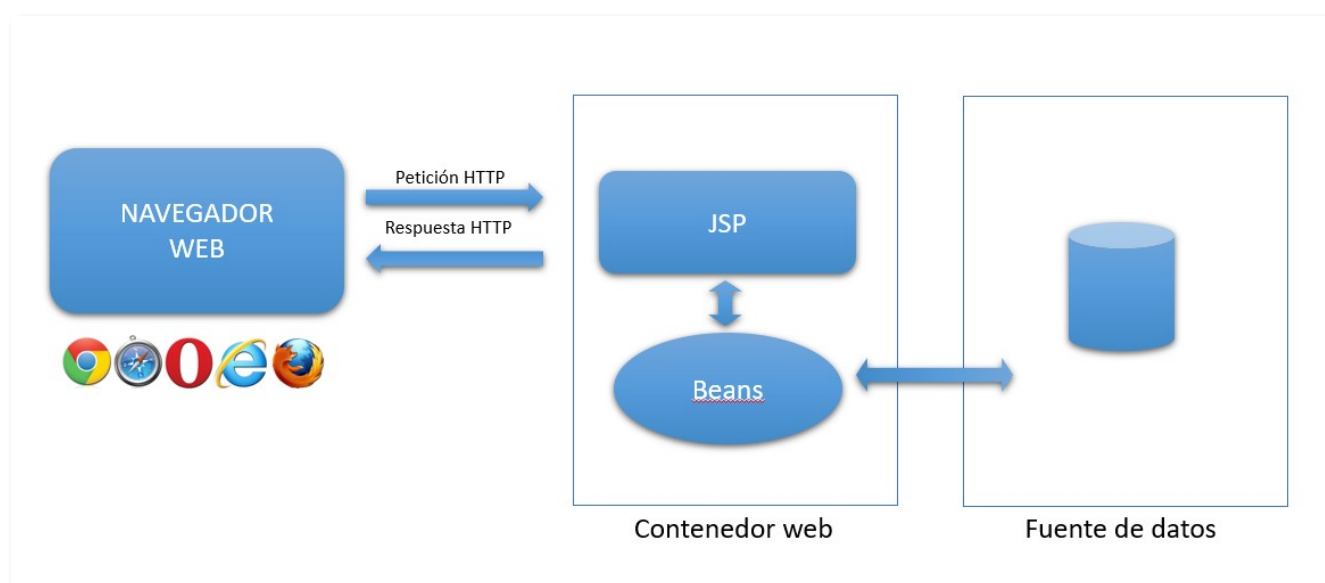
    public void doGet(HttpServletRequest request, HttpServletResponse response)
        throws IOException, ServletException
    {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<html>");
        out.println("<head>");
        out.println("<title>Hello World!</title>");
        out.println("</head>");
        out.println("<body>");
        out.println("<h1>Hello World!</h1>");
        out.println("</body>");
        out.println("</html>");
    }
}
```

Os animamos a que ejecutéis todos los ejemplos de servlets y reviséis el código asociado, ya que es muy didáctico. Podréis ver cómo se recupera información del navegador (**Request Headers**), cómo se reciben parámetros (**Request Parameters**), cómo trabajar con cookies (**Request Cookies**) y con sesiones (**Request Sessions**).

A continuación se revisarán los ejemplos de JSP, pero antes vamos a dar unas nociones básicas acerca de qué son y cómo funcionan.

3. JavaServer Pages (JSP)

JavaServer Pages (JSP) es una tecnología que permite crear páginas web dinámicas basadas en HTML, XML y XHTML. JSP es parecido PHP, pero con la diferencia de que usa el lenguaje de programación Java.



Una página JSP consta de etiquetas HTML y etiquetas JSP.

Los JSP se pueden considerar como una extensión de los Servlets, que proporcionan más funcionalidad como un lenguaje de expresión, etiquetas JSTL, etc. Realmente detrás de un JSP hay un servlet que es generado por el servidor, lo cual se denomina “compilar” el jsp. Los jsp no se compilan en cada petición sino únicamente en la primera petición o bien si es modificado, lo cual permite modificarlos “en caliente” sin necesidad de volver a desplegar la aplicación.

Las ventajas de los JSP frente a los servlets:

1) Son una extensión de los servlets.

Se dispone de todas las características de los servlets. Y además, se pueden emplear objetos, etiquetas predefinidas, lenguaje de expresión y etiquetas personalizadas que hacen que el desarrollo sea más sencillo.

2) Fáciles de mantener.

Las páginas JSP son más fáciles de mantener que los Servlets porque podemos separar la lógica de negocio y la capa de presentación. En los servlets se mezcla la lógica de negocio con la capa de presentación, mientras que los jsp están pensados para incluir únicamente la capa de presentación.

3) El desarrollo es más rápido, ya que no requiere recompilar y desplegar.

A diferencia de los servlets, los JSP se pueden modificar sin tener que volver a desplegar, agilizando el desarrollo.

4) Menos código que en los servlets.

En los JSP se emplean multitud de etiquetas o TagLib, como las etiquetas de acción, los JSTL o etiquetas personalizadas, que buscan reducir el código del JSP. Además, es posible emplear lenguajes de expresión u objetos implícitos.

Por lo tanto, un JSP es una página HTML donde se puede incrustar código en JAVA:

- Expresiones en JAVA: Se puede indicar una expresión Java por medio de las etiquetas `<%= y %>`. El resultado de la expresión se inserta en el código HTML. No son sentencias sino expresiones que devuelven un String o algún objeto que dispone de un método `toString()`.

```
<html>
<head>
  <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
  <title>Ejemplos JSP</title>
</head>
<body>
  <H1>Ejemplos de expresiones</H1>
  <!-- Mostrar la fecha y hora actual --%>
  <%= new java.util.Date().toString() %><br>

  <!-- Convertir a mayúsculas un String --%>
  <%= "Pasar a mayúsculas".toUpperCase() %><br>

  <!-- Resultado de una expresión aritmética --%>
  <%= (5+2)/(float)3 %><br>

  <!-- Generar un número aleatorio --%>
  <%= new java.util.Random().nextInt(100) %>
</body>
</html>
```

- Scriptlets o código Java incrustado: Por medio de las etiquetas `<% y %>` se puede insertar cualquier conjunto de sentencias Java, que serán ejecutadas por el servidor.

```
<html>
<head>
  <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
  <title>Ejemplos JSP</title>
</head>
```



```

<body>
  <H1>Ejemplo de scriptlet</H1>
  <%
    int numero = 7, factorial = 1;
    for(int i=numero; i>1; i--) {
      factorial *= i;
    }
  %>
  <!-- Se muestran en negrita el número y el resultado del factorial -->
  <%= "El factorial de <b>" + numero + "</b> es <b>" + factorial + "</b>" %>
</body>
</html>

```

- Objetos request, session y out. Objetos predefinidos de los JSP:
 - o request: permite obtener datos de la petición HTTP que se realiza. Por ejemplo, los parámetros GET y POST que son pasados a la página.
 - o session: contiene los datos de la sesión, permitiendo guardar y recuperar información durante la vigencia de la sesión.
 - o out: posibilita enviar información a la salida HTML que se genera.
- Declaraciones: Por medio de las etiquetas <%! y %>, se pueden declarar variables y métodos que puedes utilizar en otro lugar de la página JSP.

```

<%!
  // Aquí se pueden declarar variables o métodos
  private double calcularFactorial(int num) {
    int res = 1;
    for(int i=num; i>1; i--) {
      res *= i;
    }
    return res;
  }
%>

<html>
<head>
  <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
  <title>Cálculo del factorial</title>
</head>
<body>
  <h1>Cálculo del factorial</h1>
  <%
    double resultado = calcularFactorial(21);
    out.println("<p>Resultado: .+ resultado + "</p>");

```

```
%>
</body>
</html>
```

4. Ejemplos de JSP

Otro de los recursos que ofrece Tomcat son ejemplos de JSP. Se encuentran en **Jsp examples**, donde se distribuyen un gran número de ejemplos de jsps:

Arrancamos el servidor Tomcat y accedemos a la url:

<http://localhost:8080/examples/jsp/>

JSP Samples

This is a collection of samples demonstrating the usage of different parts of the Java Server Pages (JSP) specification. Both JSP 2.0 and JSP 1.2 examples are presented below.

These examples will only work when these pages are being served by a servlet engine; of course, we recommend [Tomcat](#). They will not work if you are viewing these pages via a "file://..." URL.

To navigate your way through the examples, the following icons will help:

-  Execute the example
-  Look at the source code for the example
-  Return to this screen

Tip: For session scoped beans to work, the cookies must be enabled. This can be done using browser options.

JSP 2.0 Examples

Expression Language

Basic Arithmetic	 Execute	 Source
Basic Comparisons	 Execute	 Source
Implicit Objects	 Execute	 Source
Functions	 Execute	 Source
Composite Expressions	 Execute	 Source

SimpleTag Handlers and JSP Fragments

Hello World Tag	 Execute	 Source
Repeat Tag	 Execute	 Source
Book Example	 Execute	 Source

Si consultáis el código fuente, podréis observar que los jsp son páginas en html con expresiones o código en JAVA embebido, que se ejecuta cuando se solicita el recurso al servidor.

Os recomendamos que consultéis los primeros ejemplos de “**Expression Language**” para que observéis como se insertan expresiones en los jsp y los **SimpleTag Handlers and JSP Fragments** donde se observa cómo ejecutar código en los jsp haciendo invocaciones a Taglibs (librerías que se incluyen como etiquetas en los jsp).

Por ejemplo, el caso de **Hello World Tag**, donde el jsp es una página sencilla en html:

```
<%@ taglib prefix="mytag" uri="/WEB-INF/jsp/jsp2-example-taglib.tld" %>
<html>
  <head>
    <title>JSP 2.0 Examples - Hello World SimpleTag Handler</title>
  </head>
  <body>
    <h1>JSP 2.0 Examples - Hello World SimpleTag Handler</h1>
    <hr>
    <p>This tag handler simply echos "Hello, World!" It's an example of
    a very basic SimpleTag handler with no body.</p>
    <br>
    <b><u>Result:</u></b>
    <mytag:helloWorld/>
  </body>
</html>
```

En la primera línea se define un taglib, con el prefijo **mytag**. Después se podrá observar cómo invoca dicho taglib con la expresión **<mytag:helloWorld/>**

Respecto a la implementación del taglib:

```
package jsp2.examples.simpletag;

import java.io.IOException;

import jakarta.servlet.jsp.JspException;
import jakarta.servlet.jsp.tagext.SimpleTagSupport;

/**
 * SimpleTag handler that prints "Hello, world!"
 */
public class HelloWorldSimpleTag extends SimpleTagSupport {
    @Override
    public void doTag() throws JspException, IOException {
        getJspContext().getOut().write( "Hello, world!" );
    }
}
```

Es una simple clase que hereda de SimpleTagSupport y sobrescribe el método doTag, escribiendo el mensaje "Hello, world!"

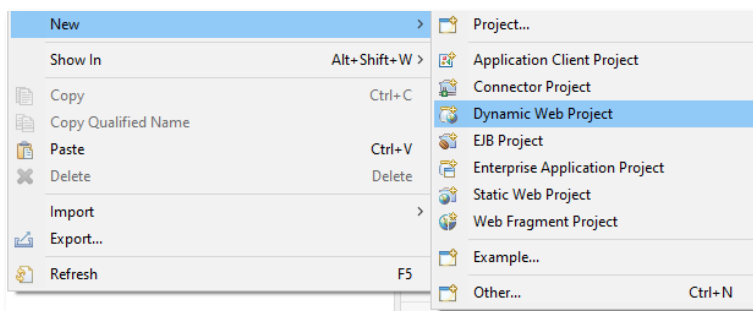
5. Cómo crear un proyecto web Dinámico

Eclipse ofrece diversos tipos de proyecto, según el tipo de aplicación que deseemos desarrollar. Entre ellos se encuentran:

- Application Client Project
- Connector Project
- Dynamic Web Project
- EJB Project

- Enterprise Application Project
- Static Web Project
- Web Fragment Project.

En este módulo vamos a centrarnos en el proyecto web dinámico. Este tipo de proyecto es muy útil para desarrollar y depurar aplicaciones web.

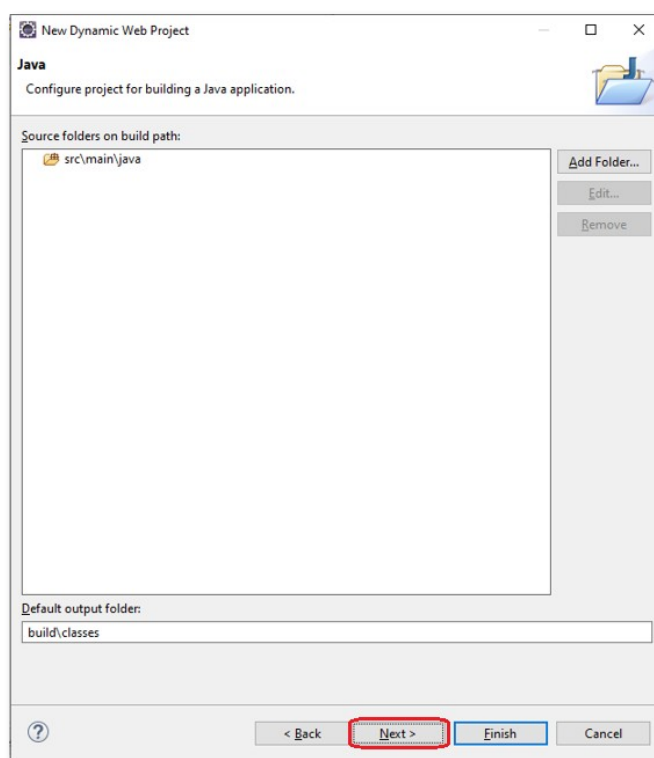
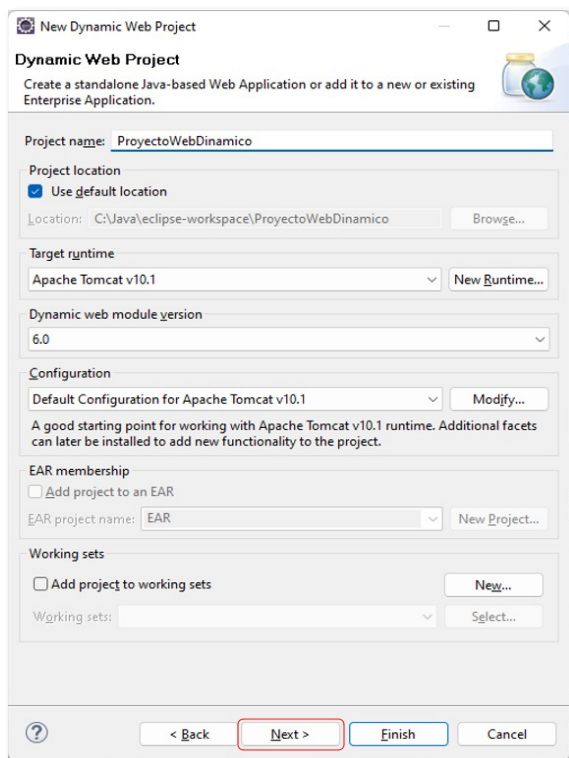


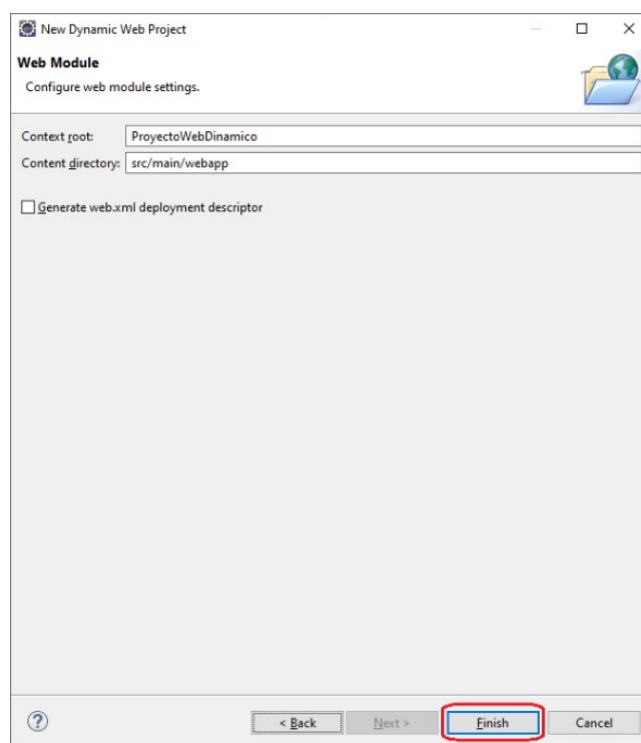
Si no aparece “Dynamic Web Project” en el menú New, se debe seleccionar New -> Other... y buscarlo ahí.

Al pinchar nos aparecerá un asistente que nos guiará en la creación del proyecto. En dicho asistente habrá que indicarle:

- El nombre del proyecto.
- En entorno de ejecución, donde se debería elegir el servidor Tomcat que hayamos creado.

El resto de los parámetros se pueden dejar con los valores por defecto.



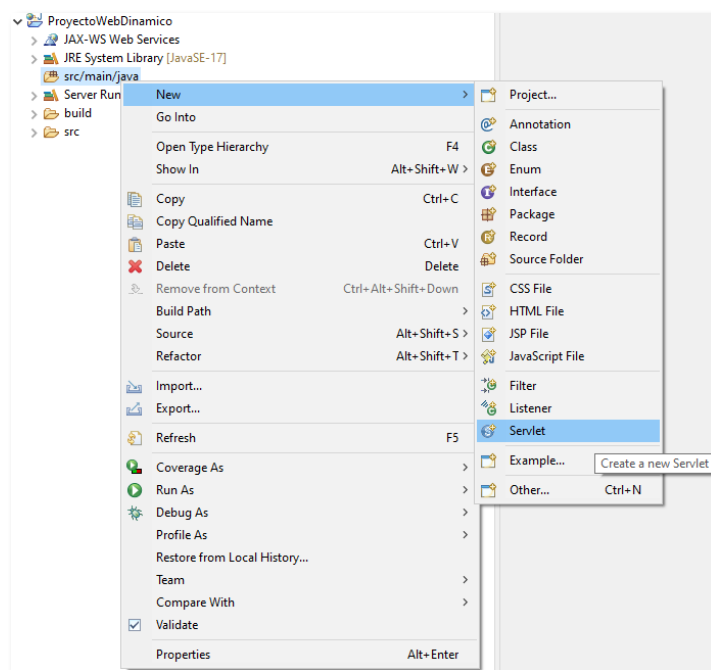


Una vez creado el proyecto, veremos que este tiene una estructura distinta a otros proyectos JAVA, aparecerán referencias a las librerías del servidor y una referencia a JAX-WS Web Services. También aparecerán las siguientes carpetas:

- src/main/java. Donde se ubicarán las clases.
- src/main/webapp. Con los ficheros de recursos que se publicarán en la aplicación web.
- build. Donde se almacenarán las clases compiladas.

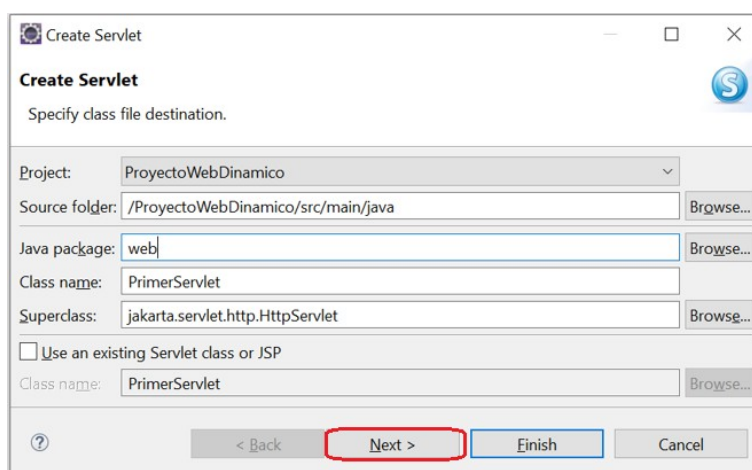
Creación de un servlet

A continuación, procederemos a crear un servlet en el directorio `src/main/java` a través del asistente disponible en Eclipse.



En dicho asistente indicaremos:

- Java package: El paquete donde deseamos crear el servlet. Le indicaremos que en el paquete *web*.
- Class Name: El nombre del servlet. *PrimerServlet*
- Parameters: Si se desea establecer algún parámetro de inicialización en el servlet, como podría ser un mensaje de error, o un parámetro de configuración del servlet. En este caso, no indicaremos ninguno.
- Los métodos que implementaremos. De momento seleccionaremos `doGet` y `doPost`.



Una vez creado el servlet veréis una clase Java con la estructura descrita anteriormente.

```
package web;

import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebInitParam;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;

/**
 * Servlet implementation class PrimerServlet
 */
public class PrimerServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;

    /**
     * @see HttpServlet#HttpServlet()
     */
    public PrimerServlet() {
        super();
        // TODO Auto-generated constructor stub
    }

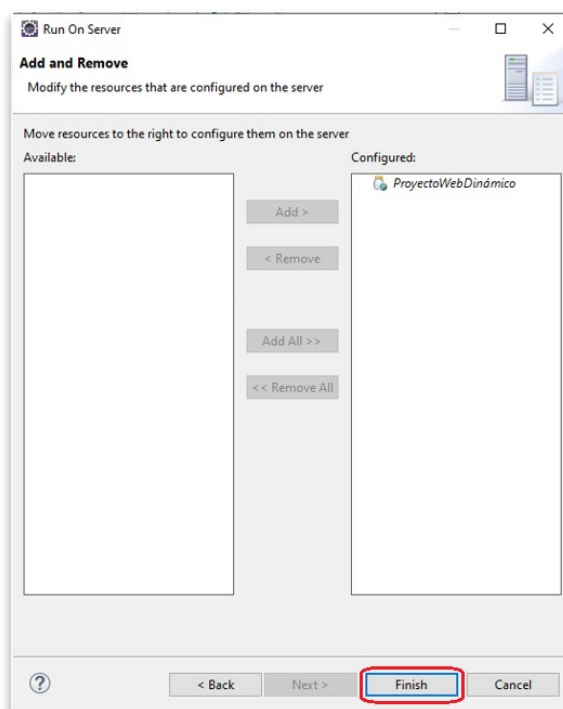
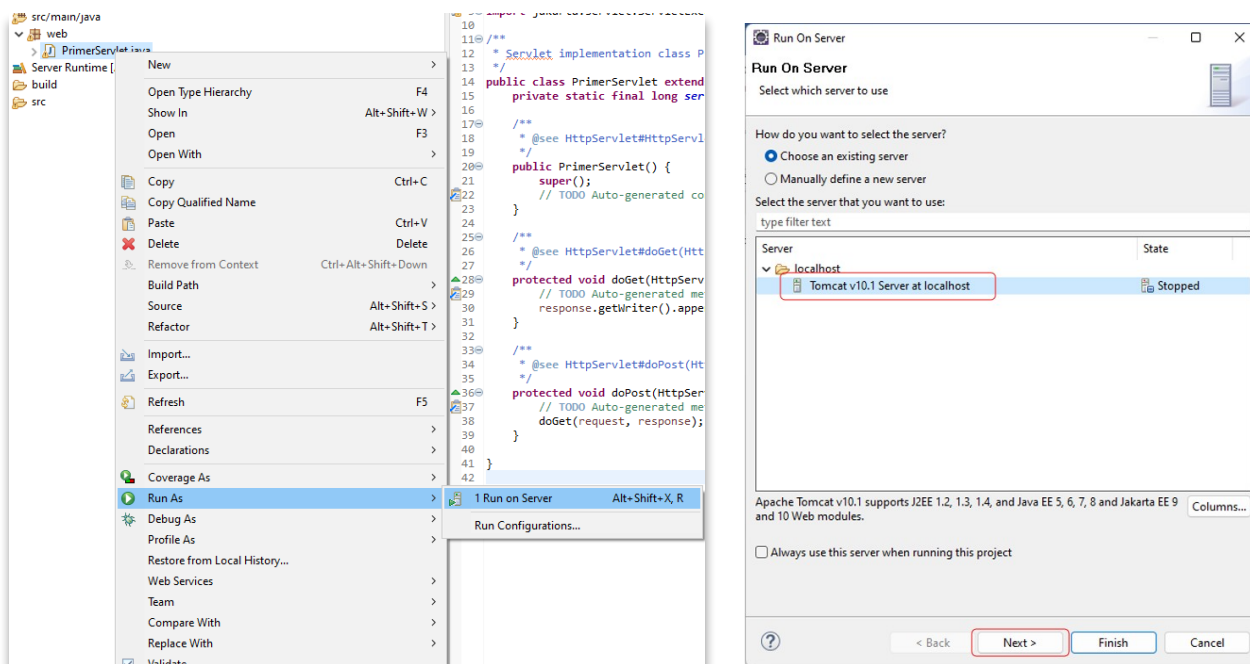
    /**
     * @see HttpServlet#doGet(HttpServletRequest request, HttpServletResponse response)
     */
    protected void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
        // TODO Auto-generated method stub
        response.getWriter().append("Served at: ").append(request.getContextPath());
    }

    /**
     * @see HttpServlet#doPost(HttpServletRequest request, HttpServletResponse response)
     */
    protected void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
        // TODO Auto-generated method stub
        doGet(request, response);
    }
}
```

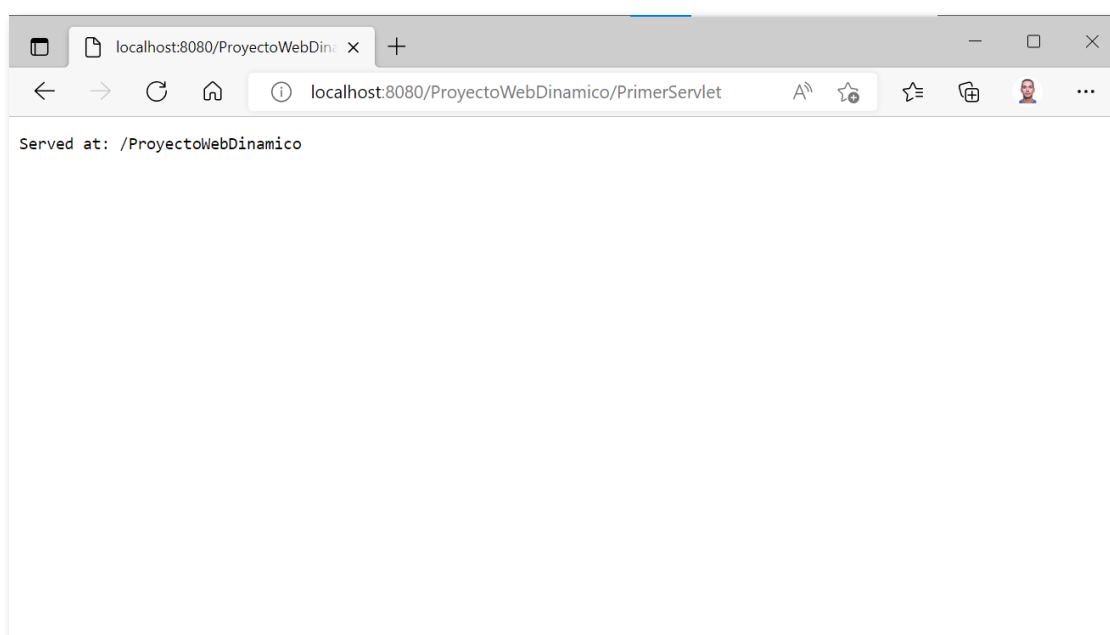
En el servlet podréis ver cómo el método doGet se limita a responder escribiendo que ha sido servido por el nombre del contexto de la aplicación.

En el caso del método doPost, invoca al método doGet sin más. Por lo tanto, la respuesta del servlet es la misma si se llama a través de un formulario (post) como si se llama a través de la url (get).

A continuación, procederemos a ejecutarlo en el servidor Tomcat. Para ello, pulsaremos el botón derecho del ratón encima del servlet, y al seleccionar la opción Run As → Run on Server le indicaremos que lo debe ejecutar en un servidor. Posteriormente indicaremos el servidor que hemos creado previamente y arrancará el servidor con el servlet.



Eclipse mostrará la siguiente pantalla:

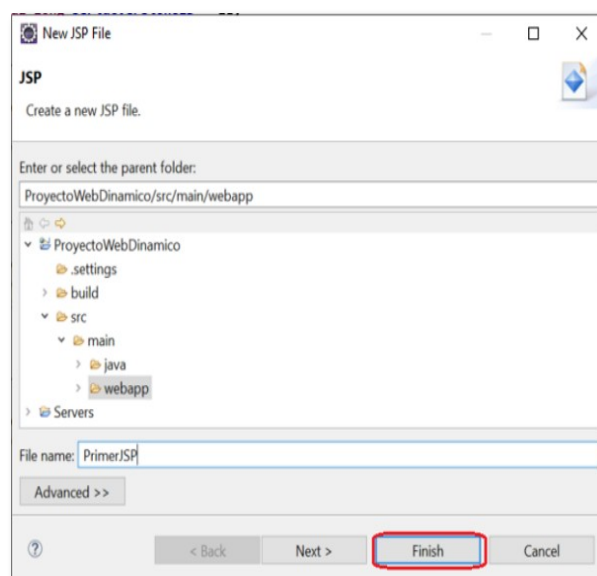
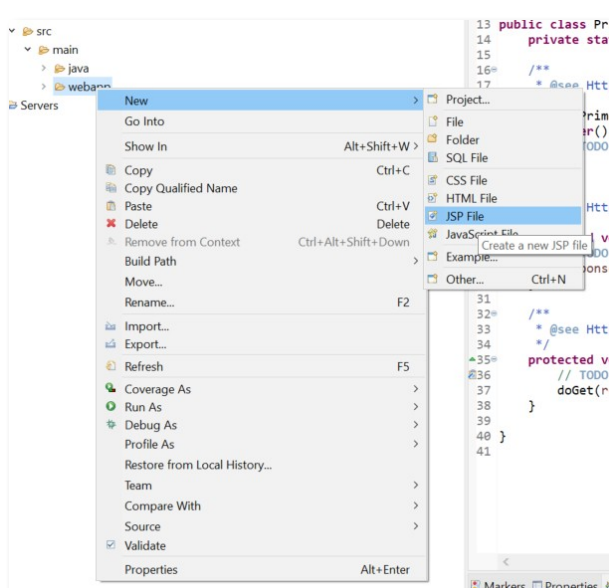


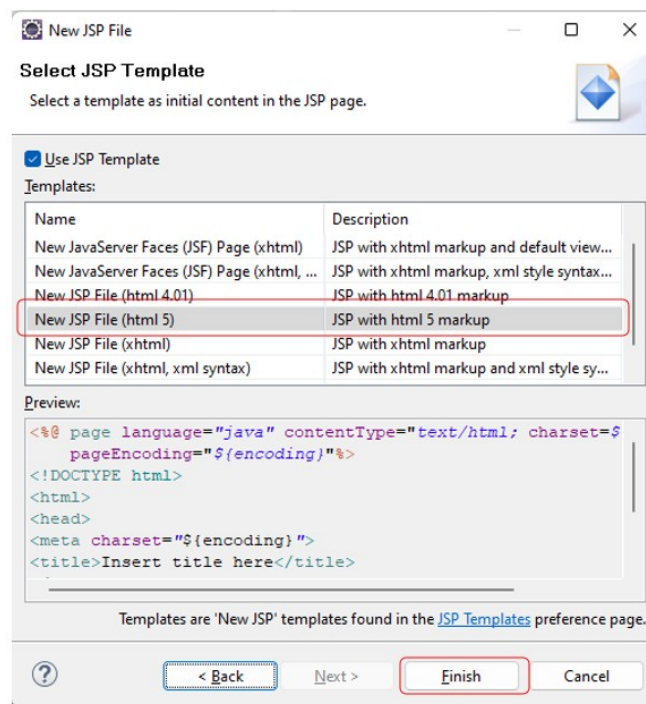
No se ha especificado cuál es la página de inicio del proyecto, por lo que, si en lugar de que pulsar sobre el servlet se pulsa sobre el proyecto, no aparecerá en el navegador la página de inicio directamente. En este caso sería necesario indicarle al navegador la url del servlet.

localhost:8080/ProyectoWebDinamico/PrimerServlet

Creación de un JSP

Para crear un JSP, debemos pulsar sobre la carpeta webapp con el botón derecho del ratón e indicarle que deseamos crear un JSP File. Posteriormente le daremos un nombre y le indicaremos la plantilla que deseamos emplear. En nuestro caso emplearemos una plantilla de JSP con HTML 5.





```
<%@ page language="java" contentType="text/html; UTF-8" %>
<!DOCTYPE html>
<html>
<head>
<title>Mi primer JSP</title>
</head>
<body>

</body>
</html>
```

Y se puede acceder al servlet por medio de la url:

<http://localhost:8080/ProyectoWebDinamico/PrimerJSP.jsp>

Ahora vamos a mostrar la hora del servidor insertando código JAVA en el JSP y veremos cómo se comporta el JSP.

```
<%@ page language="java" contentType="text/html; UTF-8" %>
<%@ page import="java.util.*" %>
<%@ page import="java.text.SimpleDateFormat" %>

<!DOCTYPE html>
<html>
<head>
<title>Mi primer JSP</title>
</head>
<body>
<%
    Date dNow = new Date();
    SimpleDateFormat ft =
    new SimpleDateFormat ("dd/MM/yyyy");
    String currentDate = ft.format(dNow);
```

```
%>
<p>La fecha actual es <%=currentDate.toString() %></p>

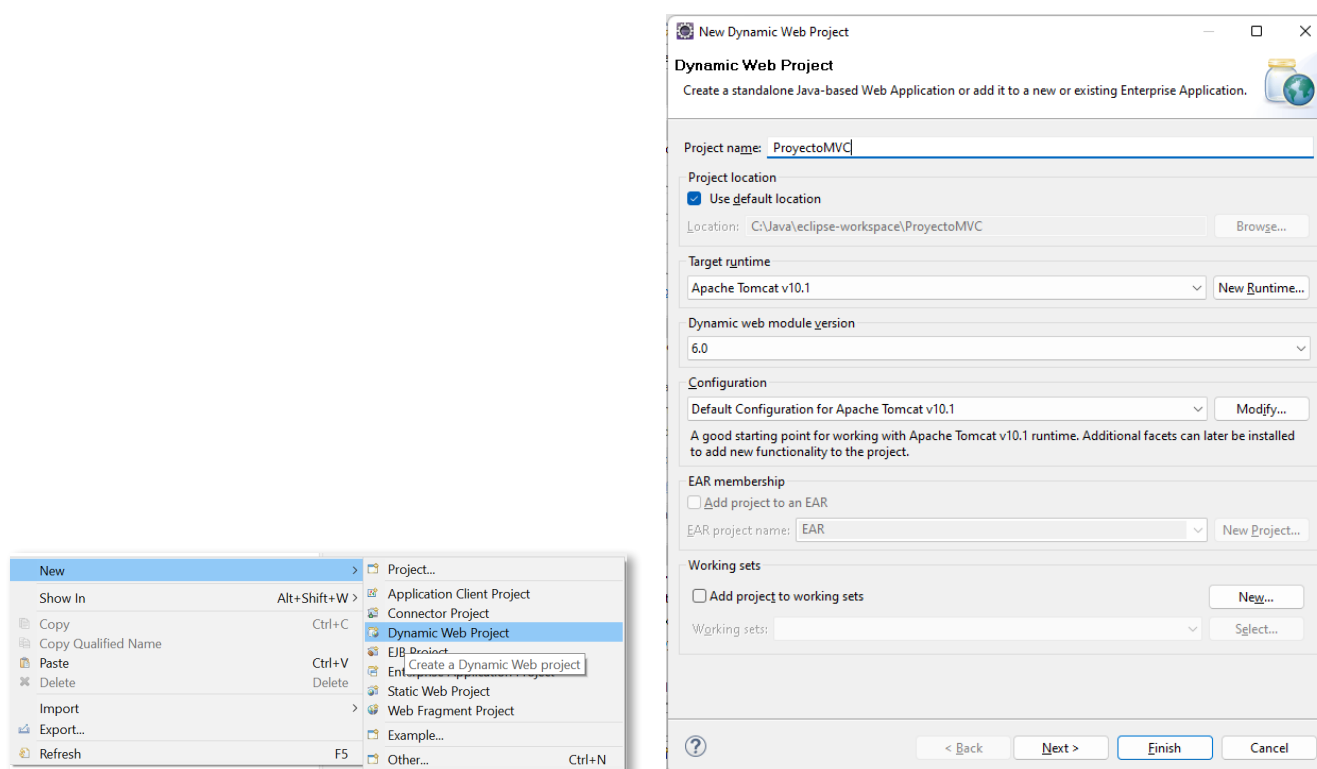
</body>
</html>
```

6. Ejemplo – MVC

Existen varios frameworks que implementan el patrón modelo-vista-controlador (MVC). Sin embargo, quedarían fuera del alcance del curso. En este ejercicio se propone implementar una pequeña aplicación web que permita guardar una lista de usuarios haciendo uso del patrón MVC. Se pueden identificar los distintos componentes del patrón:

- Vista (View): representa la interfaz de usuario, forman las páginas con las que interactúa el usuario. Se construirá por medio de JSP.
- Modelo (Model): es donde está toda la lógica del negocio, la representación de todo el sistema incluida la interacción con una base de datos si fuese necesario.
- Controlador (Controller): este componente es el que responde a la interacción (eventos) que hace el usuario en la interfaz y realiza las peticiones al modelo para pasar estos a la vista.

Creemos un proyecto web dinámico.



Una vez tengamos el proyecto, crearemos tres paquetes:

- controller
- model
- dao

En el paquete model crearemos una clase que representará el modelo de dominio con el que opera el ejemplo. En este caso, la clase se denominará Cliente.java.

```
package model;

import java.util.Date;

public class Cliente{
    private int id;
    private String nombre;
    private String apellidos;
    private Date fechaNacimiento;
    private int numPedidos;

    public Cliente(){

    }

    public int getId() {
        return id;
    }

    public String getNombre() {
        return nombre;
    }

    public String getApellidos() {
        return apellidos;
    }

    public Date getFechaNacimiento() {
        return fechaNacimiento;
    }

    public int getNumPedidos() {
        return numPedidos;
    }

    public void setId(int id) {
        this.id = id;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public void setApellidos(String apellidos) {
        this.apellidos = apellidos;
    }

    public void setFechaNacimiento(Date fechaNacimiento) {
        this.fechaNacimiento = fechaNacimiento;
    }

    public void setNumPedidos(int numPedidos) {
        this.numPedidos = numPedidos;
    }
}
```

Una vez creada la clase Cliente, implementaremos en el paquete dao una clase que simule un DAO (Data Access Object) donde se almacene la información. Es decir, simularemos una base de datos. Dicha clase se llamará ClienteDao.java

```
package dao;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import model.Cliente;

public class ClienteDao {
    private Integer id=0;
    private HashMap<Integer, Cliente> bdClientes;

    public ClienteDao() {
        bdClientes = new HashMap<Integer, Cliente>();
    }

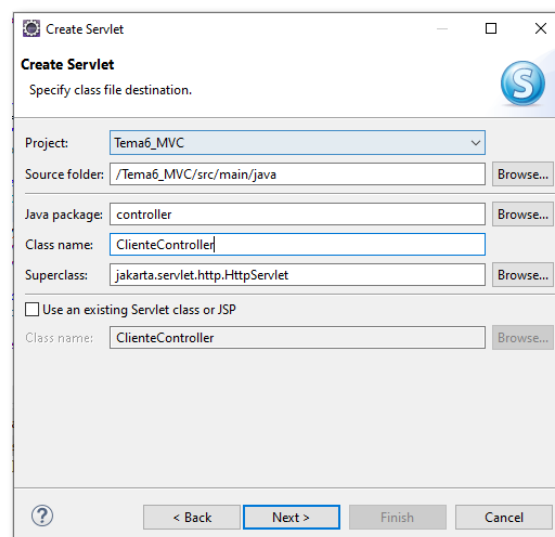
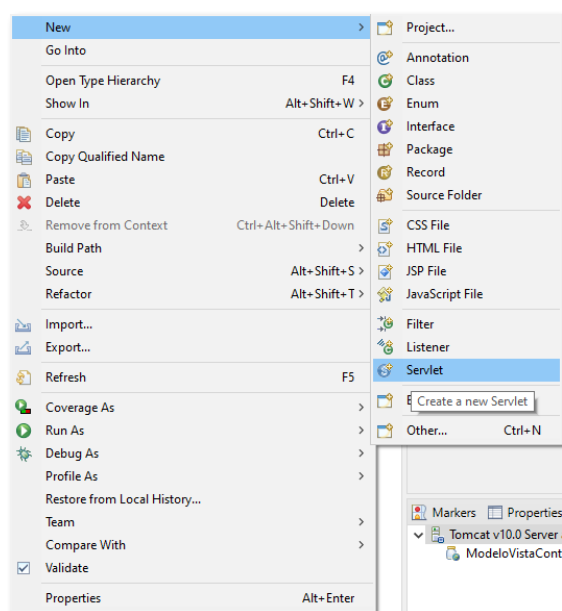
    public void insertarCliente(Cliente cliente) {
```

```

        cliente.setId(id);
        bdClientes.put(id, cliente);
        id++;
    }
    public void borrarCliente(int clienteId) {
        bdClientes.remove(clienteId);
    }
    public void actualizarCliente(Cliente cliente) {
        bdClientes.put(cliente.getId(), cliente);
    }
    public List<Cliente> obtenerClientes() {
        return new ArrayList<Cliente>(bdClientes.values());
    }
    public Cliente obtenerClienteById(int clienteId) {
        return bdClientes.get(clienteId);
    }
}

```

Posteriormente en el paquete controller añadiremos un servlet que hará las funciones de controlador. Lo llamaremos ClienteController.



Este servlet será el encargado de recibir las peticiones, realizar las acciones que correspondan de la lógica de negocio, y finalmente redirigir a la vista correspondiente. Es decir, el controlador (servlet) realizará ciertas acciones, como resultado construirá uno o varios objetos (modelo), y redirigirá al jsp (vista). El jsp tendrá acceso a los objetos del modelo que ha construido el controlador y su única función será representarlos.

```

package controller;

import java.io.IOException;
import java.text.ParseException;
import java.text.SimpleDateFormat;
import java.util.Date;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import jakarta.servlet.RequestDispatcher;
import dao.*;
import model.*;

```

```

public class ClienteController extends HttpServlet {
    private static final long serialVersionUID = 1L;
    private static String INSERTAR_O_EDITAR = "/cliente.jsp";
    private static String LISTADO = "/listadoClientes.jsp";
    private ClienteDao dao;
    public ClienteController() {
        super();
        dao = new ClienteDao();
    }
    protected void doGet(HttpServletRequest request,
        HttpServletResponse response) throws ServletException, IOException
    {
        String urlVista="";
        String action = request.getParameter("action");

        switch (action) {
            case "borrar": {
                int clienteId = Integer.parseInt(request.getParameter("clienteId"));
                dao.borrarCliente(clienteId);
                request.setAttribute("clientes", dao.obtenerClientes());
                urlVista = LISTADO;
                break;
            }
            case "editar" :{
                int clienteId = Integer.parseInt(request.getParameter("clienteId"));
                Cliente cliente = dao.obtenerClienteById(clienteId);
                request.setAttribute("cliente", cliente);
                urlVista = INSERTAR_O_EDITAR;
                break;
            }
            case "listado" : {
                request.setAttribute("clientes", dao.obtenerClientes());
                urlVista = LISTADO;
                break;
            }
            default:
                urlVista = INSERTAR_O_EDITAR;
        }
        RequestDispatcher vista = request.getRequestDispatcher(urlVista);
        vista.forward(request, response);
    }

    protected void doPost(HttpServletRequest request,
        HttpServletResponse response) throws ServletException, IOException
    {
        // Recuperamos los parámetros
        String strClienteId = request.getParameter("clienteId");
        String strNombre = request.getParameter("nombre");
        String strApellidos = request.getParameter("apellidos");
        String strFechaNacimiento = request.getParameter("fechaNacimiento");
        String strNumPedido = request.getParameter("numPedidos");

        // Creamos la clase Cliente y le asignamos los valores
        try {
            Cliente cliente = new Cliente();
            cliente.setNombre(strNombre);
            cliente.setApellidos(strApellidos);
            Date fecha = new SimpleDateFormat("dd/MM/yyyy").parse(strFechaNacimiento);
            cliente.setFechaNacimiento(fecha);
            cliente.setNumPedidos(Integer.parseInt(strNumPedido));
            if(strClienteId == null || strClienteId.isEmpty())
            {
                // Si NO tiene idCliente, es nuevo
                dao.insertarCliente(cliente);
            }
            else
            {
                // Si tiene idCliente, es una actualización
                cliente.setId(Integer.parseInt(strClienteId));
                dao.actualizarCliente(cliente);
            }
        } catch (ParseException e) {

```

```

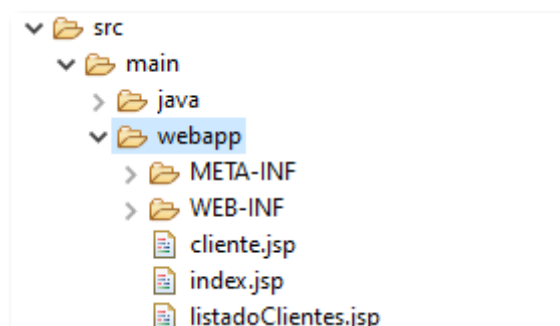
        e.printStackTrace();
    }
    RequestDispatcher vista = request.getRequestDispatcher(LISTADO);
    request.setAttribute("clientes", dao.obtenerClientes());
    vista.forward(request, response);
}
}

```

En el método doGet vemos:

- El controlador recupera un parámetro de la url recibida, mediante `action = request.getParameter("action")`. Es decir, que si la url es `/ClienteController?action=editar`, entonces la variable `action` tomará el valor "editar". De esta manera indicaremos al controlador qué acción queremos realizar.
- El controlador realiza acciones de la lógica de negocio mediante llamadas al dao: `dao.borrarCliente`, `dao.editarCliente`, etc.
- El controlador construye objetos del modelo para pasárselos al jsp. Por ejemplo, al editar:
 - `Cliente cliente = dao.obtenerClienteById(clienteId)`; → Obtiene el cliente, que es el objeto del modelo que pasaremos a la vista.
 - `request.setAttribute("cliente", cliente)`; → Mediante esta instrucción hacemos que el jsp pueda acceder al objeto cliente, para lo cual tendrá que utilizar la sintaxis `${cliente}`.
- Finalmente el controlador redirige al jsp con la instrucción `vista.forward(request, response)`, donde `vista` se ha construido a partir del nombre del jsp con el método `request.getRequestDispatcher`

Continuando con el ejemplo, crearemos las vistas en la carpeta `src/main/webapp`



En concreto, crearemos un jsp de inicio, denominado `index.jsp`, que se limita a redirigir al controlador indicando que la acción que debe ejecutar es mostrar el listado.

```

<%@ page language="java" contentType="text/html; UTF-8" %>
<html>
<head>
<title>Página inicial</title>
</head>
<body>
<jsp:forward page="/ClienteController?action=Listado" />
</body>
</html>

```

Posteriormente, crearemos la vista que mostrará un listado de clientes, que recibirá en el parámetro clientes y permitirá, actualizar dicho cliente, borrarlo o añadir un nuevo cliente. Esta vista se denominará listadoClientes.jsp

```
<%@ page language="java" contentType="text/html; charset=UTF-8" %>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c"%>
<%@ taglib uri="http://java.sun.com/jsp/jstl/fmt" prefix="fmt"%>
<html>
<head>
<title>Todos los Clientes</title>
</head>
<body>
<h4>Todos los clientes</h4>
<table border=1>
<thead>
<tr>
<th>Id</th>
<th>Nombre</th>
<th>Apellidos</th>
<th>Fecha Nacimiento</th>
<th>Pedidos</th>
<th colspan=2>Acción</th>
</tr>
</thead>
<tbody>
<c:forEach items="${clientes}" var="cliente">
<tr>
<td><c:out value="${cliente.id}" /></td>
<td><c:out value="${cliente.nombre}" /></td>
<td><c:out value="${cliente.apellidos}" /></td>
<td><fmt:formatDate pattern="dd-MMM-yyyy" value="${cliente.fechaNacimiento}" /></td>
<td><c:out value="${cliente.numPedidos}" /></td>
<td>
<a href="ClienteController?action=editar&clienteId=<c:out value="${cliente.id}" />">Actualizar</a>
</td>
<td>
<a href="ClienteController?action=borrar&clienteId=<c:out value="${cliente.id}" />">Borrar</a>
</td>
</tr>
</c:forEach>
</tbody>
</table>
<p>
<a href="ClienteController?action=insertar">Insertar Cliente</a>
</p>
</body>
</html>
```

Finalmente deberemos añadir la vista que permite cumplimentar los datos de un Cliente, llamada cliente.jsp

```
<%@ page language="java" contentType="text/html; UTF-8" %>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c"%>
<%@ taglib uri="http://java.sun.com/jsp/jstl/fmt" prefix="fmt"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
"http://www.w3.org/TR/html4/loose.dtd">
<html>
<head>
<title>Cliente</title>
</head>
<body>
<h4>Cliente</h4>
<form method="POST" action='ClienteController' name="formulario">
<input type="hidden" name="clienteId" value="<c:out value="${cliente.id}" />" />
Nombre: <input type="text" name="nombre"
value="<c:out value="${cliente.nombre}" />" />
<br/>
Apellidos: <input type="text" name="apellidos"
value="<c:out value="${cliente.apellidos}" />" />
```



```

<br/>
Fecha Nacimiento (dd/mm/yyyy): <input type="text" name="fechaNacimiento"
                                value="<fmt:formatDate pattern="dd/MM/yyyy"
                                value="${cliente.fechaNacimiento}" /> />

<br/>
Pedidos : <input type="text" name="numPedidos"
           value="<c:out value="${cliente.numPedidos}" />" />

<br/>
<input type="submit" value="Enviar" />
</form>
</body>
</html>

```

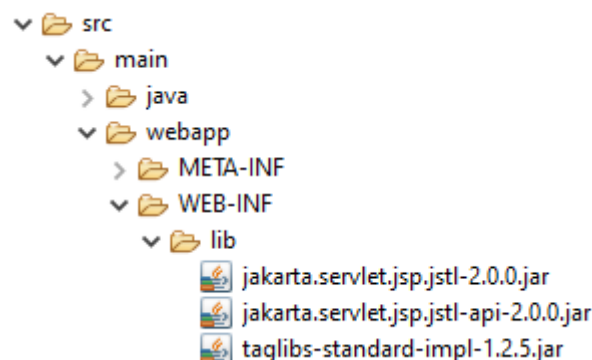
Llegados este punto veréis que eclipse genera varios errores.

✖ Can not find the tag library descriptor for "http://java.sun.com/jsp/jstl/core"	cliente.jsp	/ProyectoMVC/src/...	line 3	JSP Problem
✖ Can not find the tag library descriptor for "http://java.sun.com/jsp/jstl/core"	listadoCliente...	/ProyectoMVC/src/...	line 3	JSP Problem
✖ Can not find the tag library descriptor for "http://java.sun.com/jsp/jstl/fmt"	cliente.jsp	/ProyectoMVC/src/...	line 4	JSP Problem
✖ Can not find the tag library descriptor for "http://java.sun.com/jsp/jstl/fmt"	listadoCliente...	/ProyectoMVC/src/...	line 4	JSP Problem

El motivo es que hemos empleado unos taglib que no se han incluido entre las librerías del proyecto. En concreto:

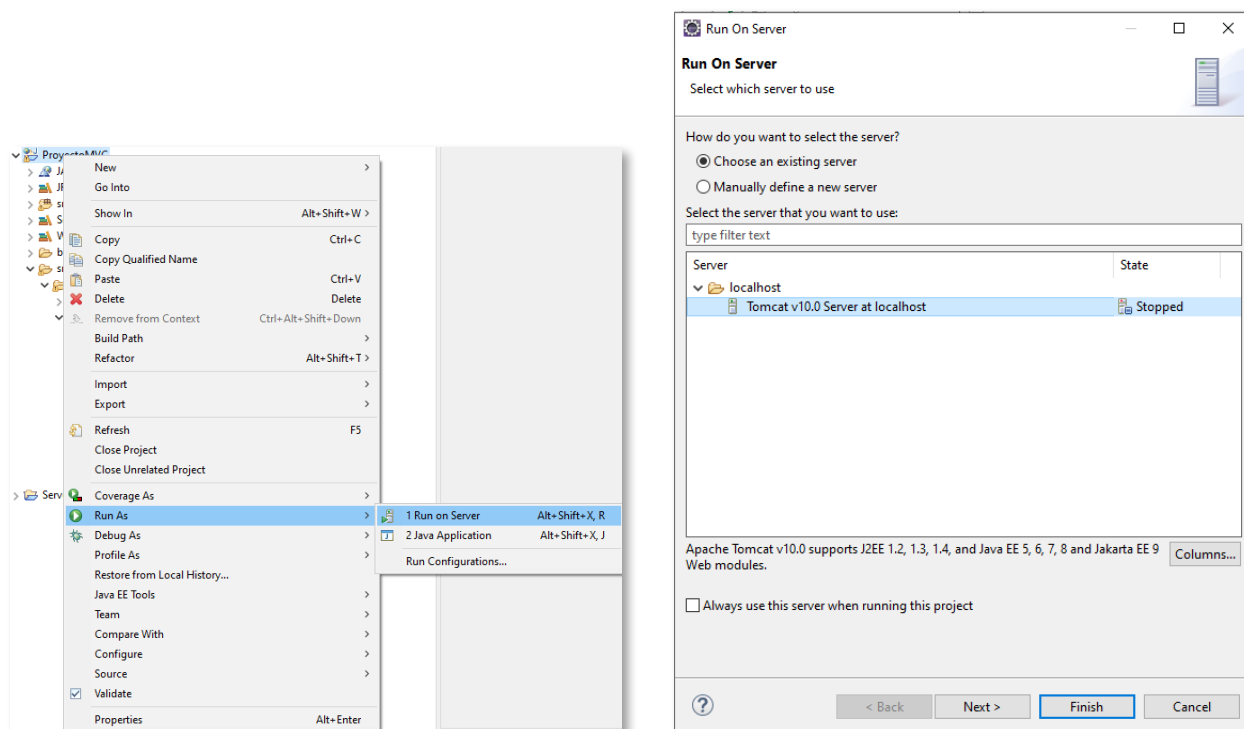
- JSTL Core. Facilitan tareas como la iteración, lógica condicional, la captura de excepciones, etc.
- JSTL Fmt. Facilitan tareas de formato de datos.

Para solventarlo basta con incluir los jar que se facilitan en el directorio webapp/WEB-INF/lib.

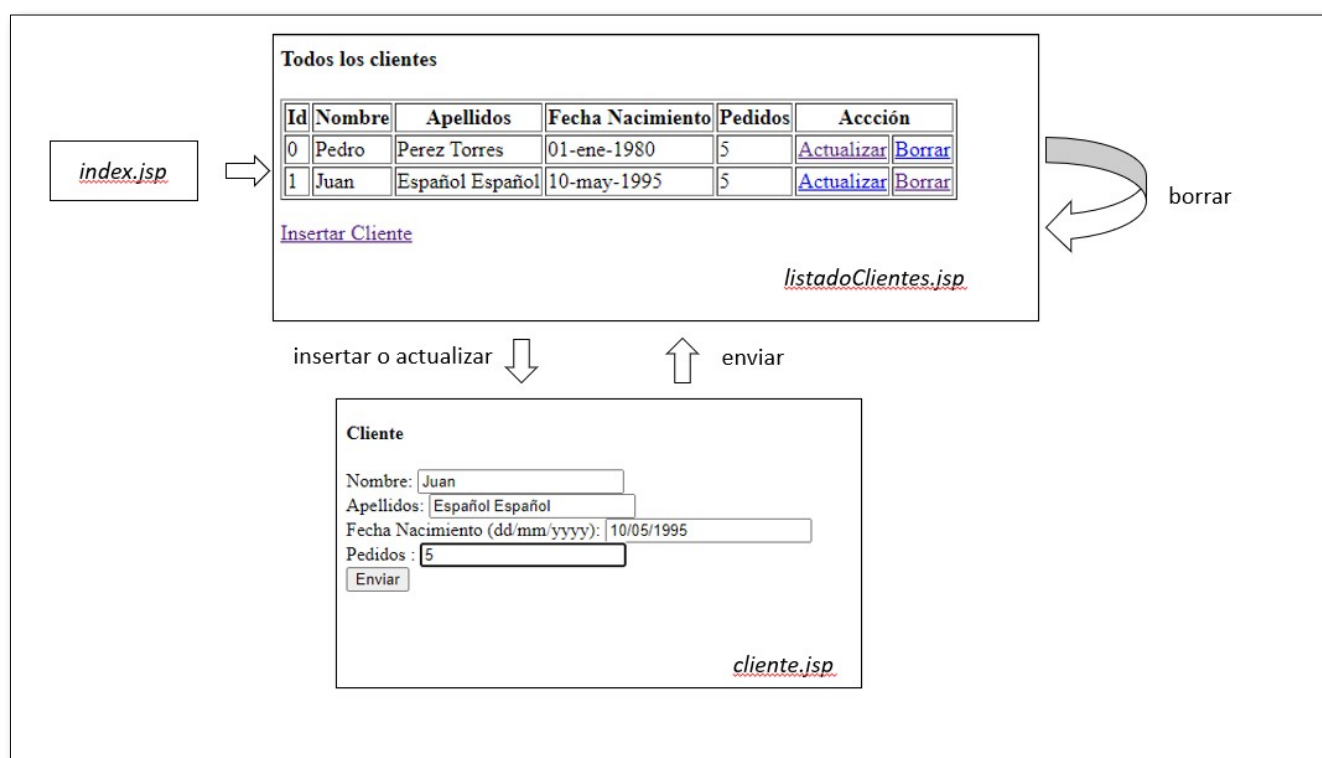


Estas librerías se pueden añadir fuera de Eclipse, en la carpeta WEB-INF/lib del proyecto, y luego desde Eclipse seleccionar Refrescar. Una vez estén estas librerías, Eclipse será capaz de interpretar los taglib. Para que Eclipse recompile el jsp deberemos modificarlo y volverlo a guardar (aunque no hagamos ningún cambio).

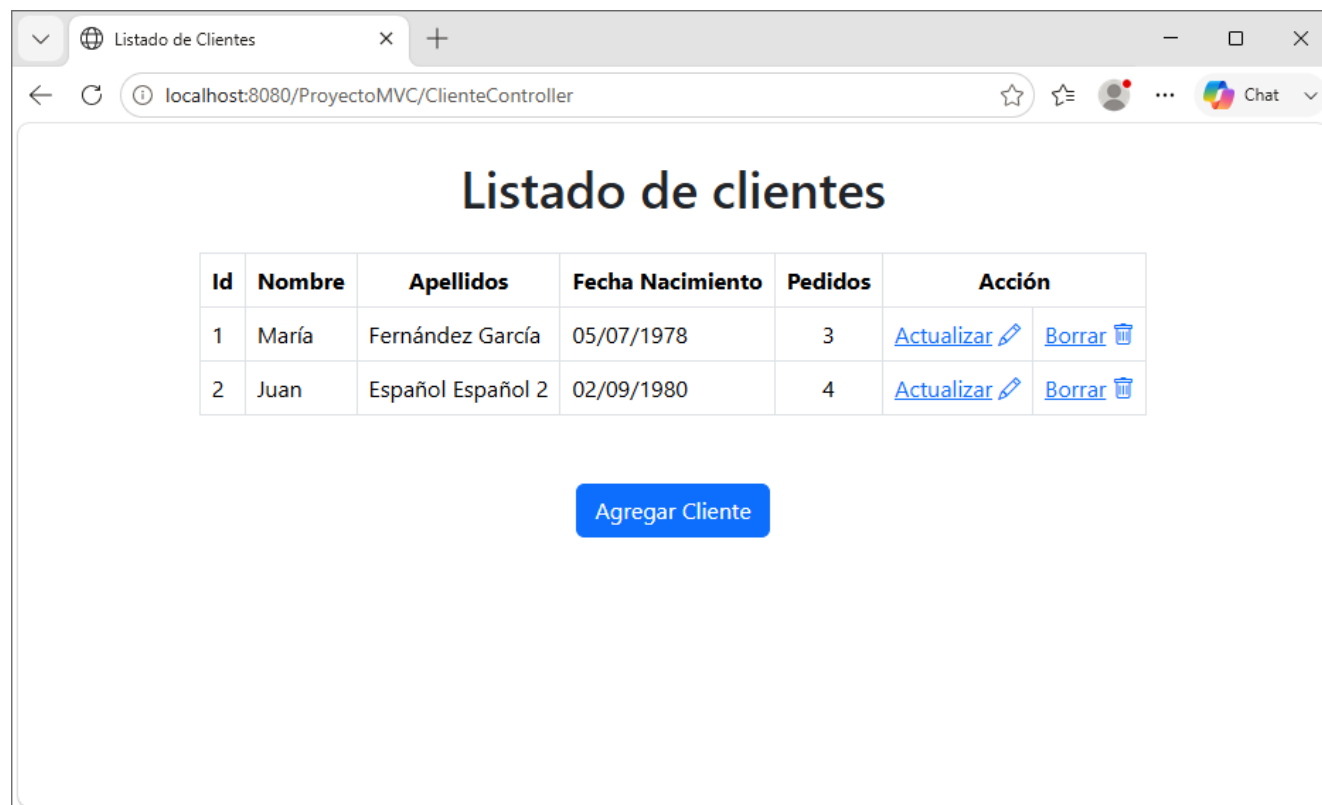
Llegados a este punto, podremos ejecutar la aplicación sobre el servidor Tomcat:



A continuación, se muestra un diagrama con el flujo entre las vistas y las acciones correspondientes:



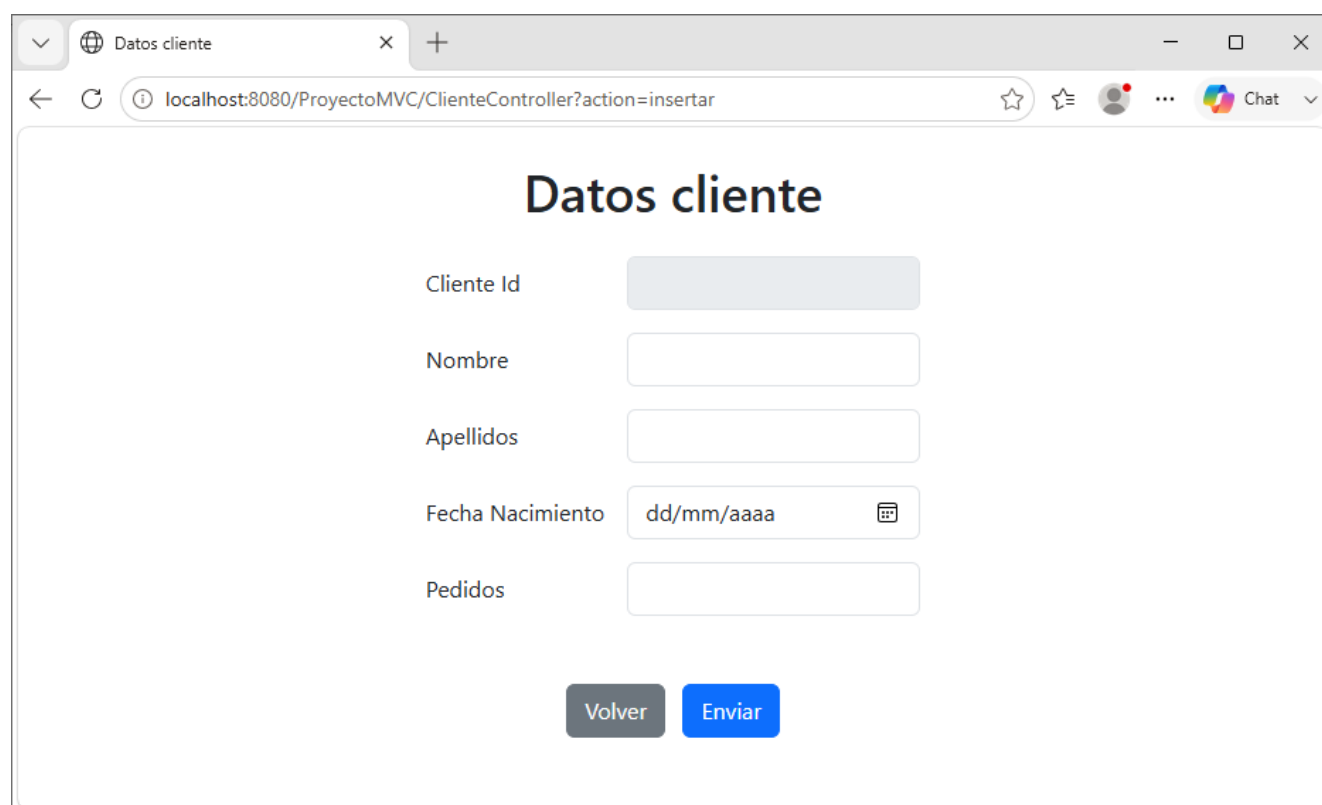
En la plataforma está subido el ejemplo completo, tanto en formato proyecto Eclipse (zip) como en formato WAR. En el ejemplo subido además se han añadido estilos utilizando la librería bootstrap, de forma que muestra este aspecto:



Listado de clientes

Id	Nombre	Apellidos	Fecha Nacimiento	Pedidos	Acción
1	María	Fernández García	05/07/1978	3	Actualizar  Borrar 
2	Juan	Español Español 2	02/09/1980	4	Actualizar  Borrar 

[Agregar Cliente](#)



Datos cliente

Cliente Id

Nombre

Apellidos

Fecha Nacimiento 

Pedidos

[Volver](#) [Enviar](#)